Herramientas de Git

Selección de revisiones

- Referencias por hash corto, nombre de rama, HEAD~n (ancestros directos) y HEAD^n (padres en fusiones).
- git log A..B muestra los *commits* en B que no están en A. git log A...B muestra los exclusivos de cada lado.
- git reflog registra los movimientos de HEAD y permite recuperar *commits* aparentemente perdidos.

Preparación interactiva

- git add -i y git add -p permiten preparar por fragmentos (*hunks*) y generar *commits* más atómicos.

Stash y limpieza

- git stash guarda temporalmente cambios sin confirmar. Se recuperan con git stash pop o git stash apply.
- git clean elimina archivos sin seguimiento, y -n permite simular antes de borrar.

Búsqueda

- git grep busca texto en el árbol de trabajo o en cualquier versión.
- git log -S localiza los *commits* que agregaron o quitaron una cadena. git log -L sigue la evolución de una función o rango de líneas.

Reescritura del historial

- git commit --amend modifica el último *commit*.
- git rebase -i permite reordenar, fusionar (*squash*), editar o descartar *commits*.
- Para reescrituras masivas se recomienda git filter-repo, que sustituye al antiguo filter-branch.

Reset desmitificado

- Git administra tres árboles: HEAD, índice y directorio de trabajo.
- --soft mueve solo HEAD, --mixed (predeterminado) también restablece el índice, y --hard modifica los tres y es destructivo.
- git revert crea un *commit* inverso sin alterar el historial publicado.

Fusión avanzada y depuración

- git merge --abort, git checkout --ours/--theirs y git rerere (reutiliza resoluciones de conflictos registradas).
- git blame indica el autor de cada línea. git bisect aplica búsqueda binaria para encontrar el *commit* que introdujo un defecto.

Submódulos

- git submodule add incluye otro repositorio fijado a un *commit* específico.

- Al clonar se usa --recurse-submodules o git submodule update --init.

Referencia

S. Chacon y B. Straub, *Pro Git*, 2.ª ed. Apress, 2014. [En línea]. Disponible en: https://git-scm.com/book/es/v2